

TRABAJO DE FIN DE GRADO

FitHero

Aplicación móvil de fitness gamificada con mecánicas de RPG

Autor: Adrián Ologu y Ángel Parro

Tutor: Damián Sualdea

Titulación: Desarrollo de Aplicaciones Multiplataforma

Convocatoria: Mayo 2026

Resumen

El presente Trabajo de Fin de Grado describe el diseño, desarrollo e implementación de FitHero, una aplicación móvil de fitness que incorpora mecánicas de gamificación propias de los videojuegos de rol (RPG) con el objetivo de fomentar hábitos de actividad física sostenibles a largo plazo.

La aplicación ha sido desarrollada en dos plataformas de forma simultánea: una versión web interactiva construida con React 18 y una versión nativa para Android implementada en Kotlin con Jetpack Compose. Ambas versiones comparten la misma lógica de negocio y sistema de diseño, lo que ha permitido validar la arquitectura propuesta en entornos tecnológicos distintos.

El sistema central de gamificación se articula en torno a tres ejes: un sistema de experiencia (XP) con progresión por niveles de curva exponencial, una economía virtual de monedas que se gana completando misiones y se gasta en una tienda in-app, y un sistema de logros que recompensa la constancia y el rendimiento del usuario. Las misiones se estructuran en cinco categorías de bienestar —fuerza, cardio, salud, hábito y mente— para ofrecer una cobertura integral de la actividad física.

Los resultados obtenidos demuestran la viabilidad técnica de la propuesta y abren una línea de trabajo hacia la integración de backend real, persistencia de datos y funcionalidades sociales que completen el producto de cara a un lanzamiento comercial.

Índice de contenidos

1. Introducción	4
1.1 Motivación y contexto	4
1.2 Objetivos	4
1.3 Estructura del documento	5
2. Estado del arte	5
3. Análisis y diseño del sistema	6
3.1 Requisitos funcionales	6
3.2 Arquitectura de la aplicación	7
3.3 Modelos de datos	8
3.4 Sistema de diseño	10
4. Implementación	11
4.1 Versión web — React	11
4.2 Versión Android — Kotlin/Compose	12
4.3 Lógica de gamificación	13
5. Resultados y pruebas	15
6. Conclusiones y trabajo futuro	16
7. Referencias	17

1. Introducción

1.1 Motivación y contexto

El sedentarismo es uno de los principales problemas de salud pública de la última década. A pesar de la abundante oferta de aplicaciones de fitness disponibles en los mercados digitales, los índices de abandono siguen siendo elevados: la mayoría de los usuarios dejan de usar estas herramientas en las primeras semanas por falta de motivación o de retroalimentación significativa.

La gamificación —entendida como la aplicación de elementos y mecánicas propias de los videojuegos en contextos no lúdicos— ha demostrado ser una estrategia eficaz para sostener la motivación intrínseca de los usuarios en aplicaciones de salud, educación y productividad. Sin embargo, pocas aplicaciones de fitness integran estas mecánicas de forma coherente y profunda; se limitan a añadir insignias o tablas de clasificación de manera superficial, sin construir un sistema de progresión que realmente involucre al usuario.

FitHero nace con el objetivo de cubrir ese hueco: una aplicación de fitness que trata el entrenamiento como una aventura de rol, donde cada sesión tiene peso narrativo y cada logro es un paso real en la evolución del personaje del usuario.

1.2 Objetivos

El objetivo general del proyecto es diseñar y desarrollar una aplicación móvil de fitness gamificada que resulte motivadora, usable y técnicamente sólida. De este objetivo general se desprenden los siguientes objetivos específicos:

- Diseñar un sistema de progresión basado en XP y niveles que mantenga al usuario comprometido a lo largo del tiempo.
- Implementar un conjunto de misiones diarias estructuradas en categorías de bienestar (fuerza, cardio, salud, hábito y mente).
- Desarrollar una economía virtual de monedas vinculada directamente al cumplimiento de objetivos.
- Crear una interfaz de usuario de alta calidad que transmita la estética y el tono del producto.
- Implementar la aplicación en dos plataformas —web con React y nativa con Kotlin/Compose— validando la portabilidad de la arquitectura propuesta.

- Sentar las bases técnicas necesarias para una futura integración con backend y lanzamiento comercial.

1.3 Estructura del documento

El documento se organiza en seis capítulos principales. Tras esta introducción, el capítulo 2 revisa el estado del arte en aplicaciones de fitness gamificadas. El capítulo 3 aborda el análisis de requisitos, la arquitectura del sistema y el diseño. El capítulo 4 detalla la implementación en ambas plataformas. El capítulo 5 presenta los resultados y las pruebas realizadas. Finalmente, el capítulo 6 recoge las conclusiones y propone líneas de trabajo futuro.

2. Estado del arte

El mercado de aplicaciones de salud y fitness ha experimentado un crecimiento sostenido en los últimos años. Aplicaciones como Nike Training Club, MyFitnessPal o Strava lideran el sector en términos de usuarios activos, aunque su enfoque es principalmente instrumental: registro de actividad, seguimiento de métricas y planificación de entrenamientos. La capa de gamificación, cuando existe, suele limitarse a rachas de actividad diaria o tablas de clasificación entre amigos.

Por otro lado, aplicaciones como Zombies, Run! o Habitica han demostrado que la integración narrativa y la mecánica de rol pueden transformar completamente la experiencia del usuario. Zombies, Run! convierte cada carrera en un episodio de una historia de supervivencia postapocalíptica, consiguiendo que el usuario se sienta parte de una narrativa más allá del simple registro de kilómetros. Habitica, aunque no centrada en el fitness, es la referencia más completa de gamificación RPG aplicada a la productividad personal.

FitHero se posiciona en el cruce de ambas corrientes: la rigurosidad en el seguimiento de actividad de las apps de fitness convencionales y la profundidad de gamificación de las apps basadas en rol. La propuesta de valor diferencial es un sistema de progresión coherente —con curva de niveles, economía virtual y logros— integrado en una experiencia visual de alta calidad diseñada desde cero.

En cuanto al stack tecnológico, la tendencia actual en el desarrollo de aplicaciones Android se orienta claramente hacia Jetpack Compose como framework de UI declarativa, en sustitución del sistema de vistas XML tradicional. Compose simplifica el desarrollo, mejora la testabilidad y se alinea con el paradigma reactivo ya establecido en el mundo web por frameworks como React. Este proyecto adopta ambas tecnologías como elecciones naturales para sus respectivas plataformas.

3. Análisis y diseño del sistema

3.1 Requisitos funcionales

Tras la fase de análisis, se identificaron los siguientes requisitos funcionales principales, organizados por módulo:

Autenticación

- El sistema debe permitir al usuario iniciar sesión con usuario y contraseña.
- Debe existir un mecanismo de autenticación biométrica (Face ID) como alternativa.
- El sistema debe registrar cada inicio de sesión para el cálculo de estadísticas de constancia.

Misiones

- El sistema debe mostrar un listado de misiones activas con su progreso actual.
- El usuario debe poder registrar incrementos y decrementos de progreso en cada misión.
- Al alcanzar el objetivo de una misión, el sistema debe otorgar automáticamente las recompensas correspondientes (XP y monedas).
- El usuario debe poder eliminar misiones del listado.

Sistema de progresión

- El sistema debe mantener un contador de XP acumulada por el usuario.
- Al superar el umbral de XP del nivel actual, el personaje debe subir de nivel automáticamente.
- El umbral de XP para el siguiente nivel debe escalar siguiendo una curva exponencial con factor 1,4.

Economía virtual

- Las monedas deben ganarse exclusivamente completando misiones.
- El usuario debe poder gastar monedas en la tienda in-app.
- El sistema debe impedir la compra si el saldo de monedas es insuficiente.

Perfil y estadísticas

- El sistema debe mostrar las estadísticas del usuario: días de actividad, calorías estimadas y entrenamientos totales.
- El usuario debe poder editar su nombre, nombre de usuario, email y foto de perfil.
- Los logros deben calcularse dinámicamente a partir del estado del usuario.

3.2 Arquitectura de la aplicación

La arquitectura del sistema sigue el patrón de flujo de datos unidireccional (UDF), ampliamente adoptado tanto en el ecosistema React como en el desarrollo Android moderno con Compose. Este patrón establece que el estado de la aplicación reside en un único punto de verdad —el componente raíz en React o el ViewModel en Android— y fluye hacia los componentes hijos de forma descendente. Las mutaciones del estado solo pueden producirse a través de eventos explícitos manejados desde la capa raíz, lo que garantiza predictibilidad y facilita la depuración.

La siguiente figura ilustra el flujo general para la acción de registrar progreso en una misión, que es representativa del funcionamiento del sistema completo:

Paso	Capa	Acción
1	UI (MissionModal)	El usuario pulsa '+ Registrar progreso'; el modal dispara <code>onUpdate(id, +1)</code>
2	Handler	<code>handleMissionUpdate</code> calcula el nuevo valor de progreso
3	Lógica de negocio	Si <code>progress ≥ goal</code> : se otorga XP, monedas y se actualizan estadísticas
4	Estado global	<code>setMissions</code> y <code>setUser</code> actualizan el estado en el componente raíz
5	Efecto secundario	<code>showToast</code> emite una notificación temporal de confirmación
6	Re-renderizado	Todos los componentes suscritos al estado se actualizan automáticamente

La versión web está contenida en un único archivo JSX por decisión deliberada: al tratarse de un prototipo funcional, la cohesión en un solo módulo facilita la portabilidad y la demostración. Esta decisión se documenta explícitamente como deuda técnica a resolver antes de cualquier despliegue en producción, donde la separación por responsabilidades en módulos independientes es obligatoria.

La versión Android sigue una estructura de paquetes estándar con separación entre capa de datos (data/Models.kt), lógica de negocio (AppViewModel.kt) y capa de presentación (ui/screens/).

3.3 Modelos de datos

A continuación se describen las entidades principales del sistema y sus atributos.

Entidad Usuario

Campo	Tipo	Descripción
name	String	Nombre completo del usuario
username	String	Identificador único (handle)
level	Int	Nivel actual del personaje
xp / xpNext	Int	XP acumulada y umbral para el siguiente nivel
coins	Int	Saldo de monedas virtuales disponibles
logins	Int	Número acumulado de inicios de sesión
role	String	Título narrativo asociado al nivel (ej. 'Héroe Novato')
weeklyActivity	List<Int>	Actividad por día de la semana (7 enteros, Dom→Sáb)
inventory	List<Int>	IDs de productos comprados en la tienda
totalCalories	Int	Estimación de calorías quemadas acumuladas
totalTrainings	Int	Número total de misiones completadas
avatar	String?	URL o cadena Base64 de la imagen de perfil

Entidad Misión

Campo	Tipo	Descripción
id	Int	Identificador único de la misión
title / desc	String	Nombre y descripción de la misión
icon / colorHex	String	Emoji representativo y color de acento (#RRGGBB)
status	String	Estado actual: 'active' o 'completed'
progress / goal	Int	Progreso actual y valor objetivo
unit	String	Unidad de medida (km, series, L, min, vez)
xpReward / coinReward	Int	Recompensas otorgadas al completar la misión
category	String	Categoría: fuerza, cardio, salud, habito o mente

Datos de inicialización

La aplicación arranca con cinco misiones predefinidas que cubren las cinco categorías de bienestar. Esta decisión permite demostrar todas las funcionalidades del sistema sin necesidad de un backend, lo que es adecuado para la fase de prototipo:

ID	Misión	Meta	XP	Monedas	Categoría
1	Entrenamiento de Fuerza	3 series	500	50	fuerza
2	Sesión de Cardio	5 km	300	30	cardio
3	Hidratación Diaria	2 L	150	15	salud
4	Madrugador	1 vez	200	20	habito
5	Meditación	10 min	180	18	mente

Logros (Achievements)

ID	Nombre	Condición de desbloqueo
primeros5km	Primeros 5km	totalTrainings $\geq$ 1
madrugador	Madrugador	logins $\geq$ 3
fuerzabruta	Fuerza Bruta	xp acumulado $\geq$ 500
kilosPerdidos	Kilos Perdidos	xp acumulado $\geq$ 2000

3.4 Sistema de diseño

Se ha desarrollado un sistema de diseño propio a partir de una paleta centrada en el color morado (#7C3AED) como tono primario. La elección responde a la voluntad de transmitir energía y ambición sin la agresividad del rojo ni la frialdad del azul. El ámbar (#F59E0B) actúa como color de acento para todo lo relacionado con recompensas y monedas, reforzando la asociación visual entre ese color y el valor. El verde (#10B981) señala los estados de éxito.

La tipografía principal es Nunito, una fuente redondeada y amigable que conecta con el tono lúdico de la aplicación sin sacrificar legibilidad. Los pesos utilizados van de Regular (400) a ExtraBold (800), lo que permite establecer una jerarquía visual clara con una sola familia tipográfica.

Los componentes reutilizables más relevantes son: MissionCard (tarjeta con icono, barra de progreso y badge de estado), ProgressBar (barra animada con relleno de color), GradientHeader (cabecera con gradiente morado y círculos decorativos

semitransparentes), y PrimaryButton (botón con gradiente, sombra de color y retroalimentación táctil).

4. Implementación

4.1 Versión web — React

La versión web está implementada como una Single Page Application en React 18 con JSX. Toda la lógica de estado reside en el componente raíz App mediante hooks nativos (useState, useEffect, useRef), siguiendo el patrón de elevación de estado (state lifting). Los handlers de mutación se definen en App y se pasan hacia abajo como props a los componentes hijos, garantizando el flujo unidireccional descrito en el apartado de arquitectura.

La estructura de componentes es la siguiente:

- LoginScreen: gestiona la autenticación mock con animación de carga de 900 ms.
- HomeScreen: dashboard principal con header de nivel y lista de MissionCard.
- MissionModal: bottom sheet con controles de progreso (+/-), barra animada y recompensas.
- ProfileScreen: perfil completo con estadísticas, logros y gráfico de actividad semanal.
- ShopScreen: tienda con tabs por categoría y grid de dos columnas.
- EditProfileModal: edición de nombre, username, email y avatar (FileReader → Base64).
- PurchaseModal: dialog de confirmación de compra con validación de saldo.
- Toast: notificación temporal de 2.200 ms posicionada en la parte superior.

Las animaciones se implementan íntegramente con CSS transitions y keyframes inline, sin dependencias externas. La transición de la barra de progreso (500 ms, ease) y la entrada del Toast (300 ms, fadeIn) son los dos efectos de mayor impacto en la percepción de fluidez de la interfaz.

La decisión de mantener todo el código en un único archivo JSX, aunque inusual, es consistente con el propósito del proyecto: un prototipo demostrable y fácilmente portable. El overhead de configurar un sistema de módulos, rutas y gestión de estado externa (Redux, Zustand) no se justifica en esta fase.

4.2 Versión Android — Kotlin/Compose

La versión Android replica la funcionalidad de la versión web con tecnología nativa. El estado de la aplicación se gestiona mediante un `AppViewModel` que expone `StateFlow` inmutables de solo lectura. Los composables consumen el estado con `collectAsState()`, lo que garantiza la recomposición automática ante cualquier cambio.

El patrón de mutación sigue el mismo principio que en React: solo el `ViewModel` puede modificar el estado, y lo hace a través de métodos públicos explícitos. Los efectos secundarios, como el `Timer` del `Toast`, se implementan con corrutinas en `viewModelScope`:

```
private val _user = MutableStateFlow(User())
val user: StateFlow<User> = _user.asStateFlow()

fun showToast(msg: String) {
    _toast.value = msg
    viewModelScope.launch { delay(2200); _toast.value = null }
}
```

Las principales diferencias de implementación respecto a la versión React son las siguientes:

Aspecto	React (Web)	Android (Compose)
Estado	<code>useState</code> hooks en componente App	<code>StateFlow</code> en <code>AppViewModel</code>
Efectos secundarios	<code>setTimeout</code> + <code>clearTimeout</code>	<code>viewModelScope.launch</code> + <code>delay</code>
Avatar de usuario	<code>FileReader</code> → cadena Base64 inline	Pendiente de implementar
Grid de tienda	CSS grid de 2 columnas	<code>chunked(2)</code> en <code>LazyColumn</code>
Animaciones	CSS transitions + keyframes	<code>animateDpAsState</code> , <code>Brush</code> gradients
Modals	<code>position: absolute</code> + <code>z-index</code>	<code>Box</code> overlay con <code>Alignment.BottomCenter</code>

Para compilar y ejecutar la versión Android se requiere Android Studio Hedgehog (2023.1.1) o superior, JDK 17 y un dispositivo o emulador con Android 8.0 (API 26) o posterior. Los emojis Unicode se renderizan correctamente a partir de esa versión de Android; en versiones anteriores pueden mostrarse como caracteres vacíos.

4.3 Lógica de gamificación

Sistema de XP y niveles

El sistema de progresión implementa una curva exponencial de escalado. Cada vez que el usuario completa una misión, su `xpReward` se suma al XP acumulado. Si el valor resultante supera el umbral `xpNext`, el personaje sube de nivel y el nuevo umbral se calcula multiplicando el anterior por un factor de 1,4:

```
newXP = user.xp + mission.xpReward
while (newXP >= xpNext) {
    newXP -= xpNext
    level += 1
    xpNext = Math.floor(xpNext * 1.4)
}
```

Con el umbral inicial de 500 XP, la progresión resultante es: Nv.1 → 500 XP · Nv.2 → 700 XP · Nv.3 → 980 XP · Nv.4 → 1.372 XP · Nv.5 → 1.920 XP. Esta curva permite que los primeros niveles sean alcanzables en pocos días de uso, generando el engagement inicial necesario, mientras que los niveles más altos requieren una dedicación sostenida en el tiempo.

Sistema de monedas

Las monedas se obtienen exclusivamente completando misiones. No existe ningún mecanismo de pérdida ni caducidad, lo que elimina la fricción negativa que puede asociarse a las economías virtuales en apps de salud. El ratio XP:monedas es constante (10:1) en todas las misiones del sistema base, lo que simplifica el diseño de precios en la tienda y garantiza que la economía sea predecible para el usuario.

Misiones y estados

Cada misión mantiene un estado binario (`active/completed`) y un contador de progreso que puede incrementarse o decrementarse dentro del rango `[0, goal]`. La posibilidad de decrementar el progreso es una decisión de diseño deliberada: permite al usuario corregir errores de registro sin penalización. Al alcanzar el valor objetivo, la misión transita automáticamente a `completed` y las recompensas se otorgan exactamente una vez, controlado mediante la comparación del estado previo (`flag wasCompleted` implícito en la comparación `status !== 'completed'`).

Logros

Los logros se calculan en tiempo real como funciones puras del estado del usuario, sin almacenarse como flags independientes. Este enfoque es eficiente y elimina la

necesidad de sincronización entre el estado de logros y el estado del usuario. La contrapartida es que no es posible detectar el instante exacto de desbloqueo para emitir un evento de animación celebratoria; esta limitación está documentada como mejora pendiente en el roadmap.

Actividad semanal

El vector `weeklyActivity` almacena un entero por cada día de la semana (índices 0-6, correspondientes a domingo-sábado, alineados con el método `getDay()` de JavaScript). Se incrementa en dos momentos: al iniciar sesión y al completar una misión. El resultado es un histograma que refleja tanto la constancia de acceso a la app como la productividad en términos de misiones completadas.

5. Resultados y pruebas

La evaluación del proyecto se ha realizado en tres dimensiones: corrección funcional, calidad de la interfaz de usuario y viabilidad técnica de la arquitectura propuesta.

Corrección funcional

Se han verificado manualmente todos los flujos principales de la aplicación: autenticación, registro de progreso en misiones, completado con otorgamiento de recompensas, compra en tienda, edición de perfil y visualización de estadísticas. Todos los flujos funcionan correctamente en ambas plataformas y producen resultados consistentes entre sí, lo que valida la coherencia de la lógica de negocio compartida.

Se han probado específicamente los casos límite del sistema de XP: subida de múltiples niveles en una sola misión (cuando el xpReward supera el umbral actual), decremento de progreso hasta cero y completado de todas las misiones disponibles.

Interfaz de usuario

La interfaz ha sido evaluada informalmente por un grupo reducido de usuarios potenciales. Las valoraciones destacan positivamente la claridad visual del sistema de progresión, la coherencia de la paleta de colores y la fluidez de las animaciones de las barras de XP y progreso. Como área de mejora se señala la necesidad de un onboarding que explique el sistema de juego a usuarios sin experiencia previa con mecánicas RPG.

Viabilidad arquitectónica

La implementación simultánea en React y Kotlin/Compose ha confirmado que la lógica de negocio es completamente independiente de la plataforma y puede portarse sin modificaciones conceptuales. Las diferencias de implementación se reducen a la sintaxis y las APIs nativas de cada entorno, pero el modelo mental subyacente —estado centralizado, flujo unidireccional, handlers explícitos— es idéntico en ambas versiones.

Limitaciones identificadas

Las limitaciones más relevantes de la versión actual son la ausencia de persistencia entre sesiones (el estado se pierde al cerrar la aplicación), la autenticación mock sin validación real de credenciales, y la falta de confirmación al eliminar una misión. Todas estas limitaciones son conocidas y están contempladas en el roadmap de la siguiente iteración.

6. Conclusiones y trabajo futuro

6.1 Conclusiones

El presente trabajo ha demostrado la viabilidad técnica y conceptual de una aplicación de fitness gamificada con mecánicas RPG. FitHero implementa un sistema de progresión coherente —XP, niveles, monedas, logros y actividad semanal— integrado en una interfaz de usuario de alta calidad, desarrollada de forma nativa en dos plataformas distintas con una arquitectura común.

La decisión de implementar el prototipo en React y Kotlin/Compose de forma simultánea ha aportado un valor añadido inesperado: ha permitido comparar los patrones de desarrollo en ambos ecosistemas y confirmar que el paradigma reactivo con flujo de datos unidireccional es igualmente aplicable y beneficioso en los dos entornos. Esta experiencia tiene un valor formativo considerable más allá del producto en sí.

Desde el punto de vista del diseño de producto, el proyecto ha evidenciado que la gamificación bien integrada —como sistema de progresión coherente, no como capa de insignias superficiales— puede aportar una diferenciación real respecto a las aplicaciones de fitness existentes en el mercado.

6.2 Trabajo futuro

Las líneas de trabajo más relevantes para la siguiente iteración del proyecto se agrupan en tres bloques:

Infraestructura técnica (alta prioridad)

- Integración de autenticación real mediante Firebase Auth o un backend propio con JWT, incluyendo login con Google y Apple.
- Implementación de persistencia de datos con Room DB en Android y AsyncStorage en React Native, para preservar el estado entre sesiones.
- Desarrollo de una API REST que centralice la lógica de negocio y permita funcionalidades multidispositivo.

Funcionalidades de producto (media prioridad)

- Creación de misiones personalizadas por el usuario, con meta, unidad y recompensa configurables.
- Sistema de racha de días (streak) con detección de actividad en el día anterior.

- Notificaciones push para recordatorios de misiones pendientes a hora personalizable.
- Pantalla de inventario con descripción extendida de los productos adquiridos.
- Animaciones de subida de nivel y desbloqueo de logros con efecto celebratorio.

Mejoras de experiencia de usuario

- Onboarding de 3-4 pantallas para usuarios nuevos que explique el sistema de juego.
- Filtrado y ordenación de misiones por categoría, recompensa y progreso.
- Modo oscuro con paleta adaptada.
- Confirmación explícita antes de eliminar una misión de forma permanente.

En cuanto a la monetización, si el proyecto evolucionara hacia un producto comercial, el modelo recomendado se basaría en cosmética pura —temas de color, avatares y títulos de rol sin ventaja de juego— y en una suscripción premium con misiones exclusivas y estadísticas avanzadas, evitando los modelos pay-to-win que generan rechazo en la comunidad de usuarios de aplicaciones de salud.

7. Referencias

Deterding, S., Dixon, D., Khaled, R. y Nacke, L. (2011). From game design elements to gamefulness: defining gamification. En Proceedings of the 15th International Academic MindTrek Conference (pp. 9-15). ACM.

Hamari, J., Koivisto, J. y Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. En 47th Hawaii International Conference on System Sciences (pp. 3025-3034). IEEE.

Google LLC. (2024). Jetpack Compose documentation. Android Developers. <https://developer.android.com/jetpack/compose>

Meta Platforms, Inc. (2024). React documentation. <https://react.dev>

World Health Organization. (2022). Physical activity. WHO Fact Sheets. <https://www.who.int/news-room/fact-sheets/detail/physical-activity>